

ITS4243 - MICROSERVICES AND CLOUD COMPUTING

Faculty of Technology - University of Sri Jayewardenepura

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science

T1. The Emergence of Microservices Architecture

Agenda

1. Introduction to Software Architectures
2. Evolution: Monolithic → SOA → Microservices
3. Characteristics of Microservices Architecture
4. Real-world Drivers for Microservices
5. Case Studies: Netflix, Amazon, Uber
6. Summary and Assignment

Introduction to Software Architectures

- Software architecture defines **the structure, organization, and interaction** of components within a system.
- It serves as a **blueprint** guiding how software is built, deployed, and maintained.
- A good architecture ensures **scalability, maintainability, and performance**.
- Three main paradigms have shaped modern software systems:
 - Monolithic Architecture
 - Service-Oriented Architecture (SOA)
 - Microservices Architecture

Architecture Style	Description	Example Use Cases
Layered (n-tier)	Organizes the system into layers (e.g., presentation, business, data). Each layer has a clear responsibility.	Web applications, enterprise systems
Client–Server	Divides system into client(s) that request services and a central server that provides them.	Banking systems, web browsers
Event-Driven	Components communicate through asynchronous events. Highly decoupled and scalable.	IoT systems, trading platforms
Service-Oriented Architecture (SOA)	Services are reusable and communicate through an Enterprise Service Bus (ESB).	Enterprise integrations, legacy modernization
Microservices Architecture	Application split into small, independent services focused on single business capabilities.	Netflix, Amazon, Uber
Serverless Architecture	Developers focus only on business logic while the cloud provider handles scaling and infrastructure.	Cloud-based APIs, data processing pipelines
Peer-to-Peer (P2P)	Each node acts as both client and server, sharing resources directly.	Torrent systems, blockchain networks

Why Architecture Matters?

- Ensures **clear separation of concerns** (each part does one thing well).
- Helps teams **scale development and operations**.
- Reduces technical debt and simplifies updates.
- Enables **technology evolution** (e.g., from on-premise to cloud).
- Architecture decisions affect **cost, performance, and reliability**.

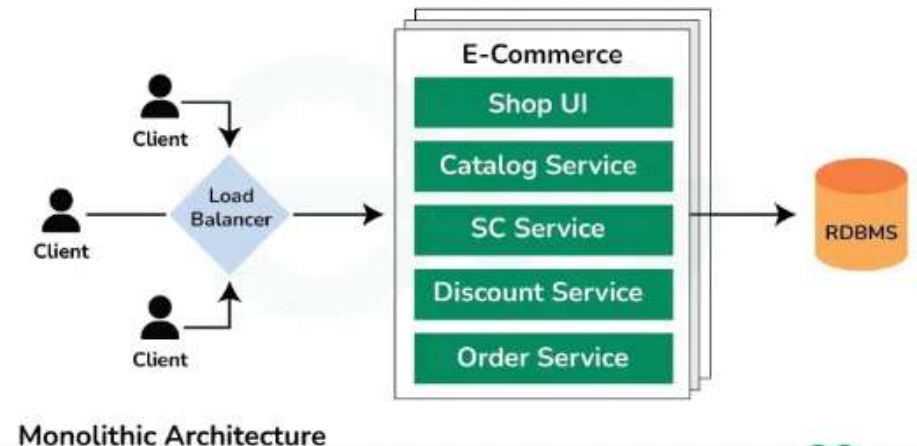
Evolution: Monolithic → SOA → Microservices

Architecture	Structure	Communication	Pros	Cons
Monolithic	Entire application built as one unit with all modules tightly integrated.	Internal function calls within same process.	Simple development & deployment (good for small apps).	Hard to scale; any small change requires redeployment of whole app; tight coupling.
SOA (Service-Oriented Architecture)	System divided into coarse-grained services that share data via an Enterprise Service Bus (ESB) .	XML/HTTP via ESB (central message broker).	Promotes reuse and integration across enterprise systems.	ESB becomes a bottleneck; heavy protocols; limited scalability.
Microservices	Application broken into fine-grained, independent services each focused on one business capability.	Lightweight communication (HTTP REST, gRPC, messaging).	Highly scalable, independent deployment, technology flexibility.	Complex to manage, requires DevOps maturity and monitoring.



Monolithic Architecture

- Monolithic architecture is a software design methodology that combines all of an application's components into a single, inseparable unit.
- Under this architecture, the user interface, business logic, and data access layers are all created, put into use, and maintained as one, unified unit.



- Simple to develop initially but difficult to scale and maintain.
- Tight coupling makes updates and deployments risky.

Example: Early e-commerce or ERP systems.

- It was preferred for its simplicity and ease of initial setup.
- Because of its rigidity, it is difficult to scale and maintain, which makes it difficult to adjust to changing needs.

Importance of Monolithic Systems

- Simplicity
- Cost-Effectiveness
- Performance
- Security
- Legacy Support

Characteristics of Monolithic Architecture

- Single Codebase
- Tight Coupling
- Shared Memory
- Centralized Database
- Limited [Scalability](#)
- Layered Structure

Key Components of Monolithic Architecture

- User Interface (UI)
- Application Logic
- Data Access Layer
- Database
- External Dependencies
- Middleware

Design Principles of Monolithic Systems

- Modularity
- Separation of Concerns
- [Scalability](#)
- Encapsulation
- [Consistency](#)

Challenges in deploying Monolithic Architecture

- Long Deployment Cycles
- Risk of Downtime
- Limited Scalability
- Resource Consumption
- Limited Flexibility

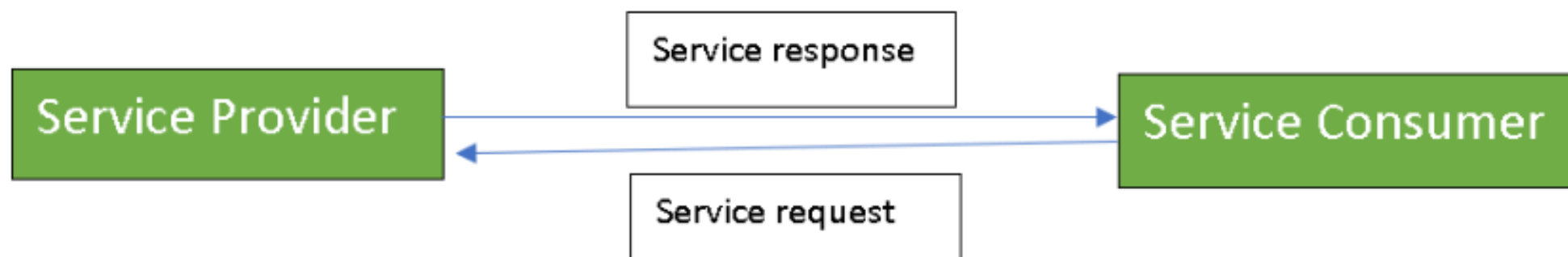
Service-Oriented Architecture (SOA)

- Service-Oriented Architecture (SOA) is a stage in the evolution of application development and/or integration. It defines a way to make software components reusable using the interfaces.
- Formally, SOA is an architectural approach in which applications make use of services available in the network. In this architecture, services are provided to form applications, through a network call over the internet. It uses common communication standards to speed up and streamline the service integrations in applications. Each service in SOA is a complete business function in itself. The services are published in such a way that it makes it easy for the developers to assemble their apps using those services. Note that SOA is different from microservice architecture.

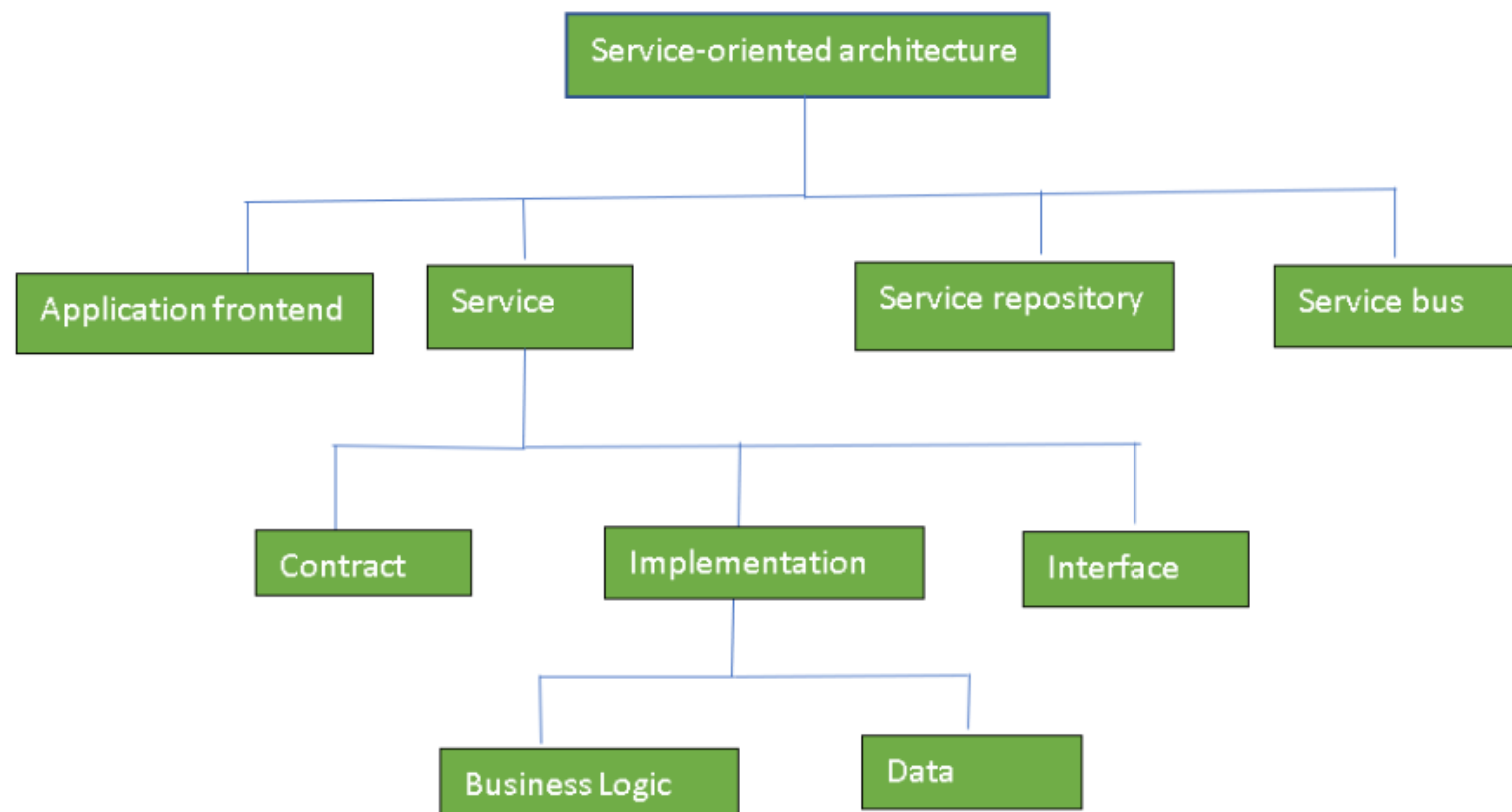
- SOA allows users to combine a large number of facilities from existing services to form applications.
- SOA encompasses a set of design principles that structure system development and provide means for integrating components into a coherent and decentralized system.
- SOA-based computing packages functionalities into a set of interoperable services, which can be integrated into different software systems belonging to separate business domains.

- **The different characteristics of SOA are as follows :**
- Provides interoperability between the services.
- Provides methods for service encapsulation, service discovery, service composition,
- service reusability and service integration.
- Facilitates QoS (Quality of Services) through service contract based on Service Level Agreement (SLA).
- Provides loosely couples services.
- Provides location transparency with better scalability and availability.
- Ease of maintenance with reduced cost of application development and deployment.

- There are two major roles within Service-oriented Architecture:
 - 1.Service provider:** The service provider is the maintainer of the service and the organization that makes available one or more services for others to use. To advertise services, the provider can publish them in a registry, together with a service contract that specifies the nature of the service, how to use it, the requirements for the service, and the fees charged.
 - 2.Service consumer:** The service consumer can locate the service metadata in the registry and develop the required client components to bind and use the service.



- Services might aggregate information and data retrieved from other services or create workflows of services to satisfy the request of a given service consumer. This practice is known as service orchestration. Another important interaction pattern is service choreography, which is the coordinated interaction of services without a single point of control.



- **Guiding Principles of SOA:**

1. **Standardized service contract:** Specified through one or more service description documents.
2. **Loose coupling:** Services are designed as self-contained components, maintain relationships that minimize dependencies on other services.
3. **Abstraction:** A service is completely defined by service contracts and description documents. They hide their logic, which is encapsulated within their implementation.
4. **Reusability:** Designed as components, services can be reused more effectively, thus reducing development time and the associated costs.
5. **Autonomy:** Services have control over the logic they encapsulate and, from a service consumer point of view, there is no need to know about their implementation.
6. **Discoverability:** Services are defined by description documents that constitute supplemental metadata through which they can be effectively discovered. Service discovery provides an effective means for utilizing third-party resources.
7. **Composability:** Using services as building blocks, sophisticated and complex operations can be implemented. Service orchestration and choreography provide a solid support for composing services and achieving business goals.

- **Advantages of SOA:**

- **Service reusability:** In SOA, applications are made from existing services. Thus, services can be reused to make many applications.
- **Easy maintenance:** As services are independent of each other they can be updated and modified easily without affecting other services.
- **Platform independent:** SOA allows making a complex application by combining services picked from different sources, independent of the platform.
- **Availability:** SOA facilities are easily available to anyone on request.
- **Reliability:** SOA applications are more reliable because it is easy to debug small services rather than huge codes
- **Scalability:** Services can run on different servers within an environment, this increases scalability

- **Disadvantages of SOA:**

- **High overhead:** A validation of input parameters of services is done whenever services interact this decreases performance as it increases load and response time.
- **High investment:** A huge initial investment is required for SOA.
- **Complex service management:** When services interact they exchange messages to tasks. the number of messages may go in millions. It becomes a cumbersome task to handle a large number of messages.

- **Practical applications of SOA:**

- SOA is used in many ways around us whether it is mentioned or not.

- 1.SOA infrastructure is used by many armies and air forces to deploy situational awareness systems.

- 2.SOA is used to improve healthcare delivery.

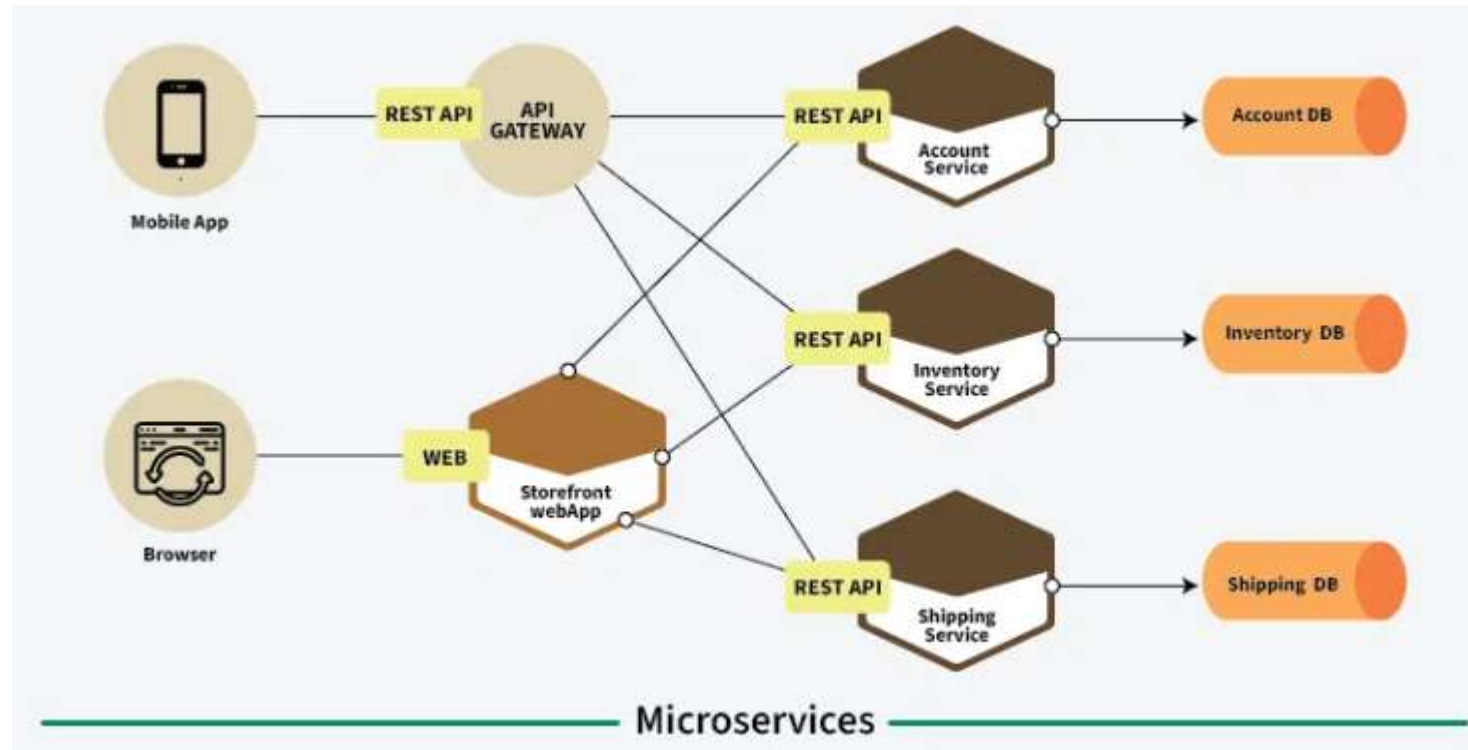
- 3.Nowadays many apps are games and they use inbuilt functions to run.
For example, an app might need GPS so it uses the inbuilt GPS functions of the device. This is SOA in mobile solutions.

- 4.SOA helps maintain museums a virtualized storage pool for their information and content.

Microservices Architecture

Microservices are an architectural approach to developing software applications as a collection of small, independent services that communicate with each other over a network.

Each service focuses on a single business capability.



Instead of building a monolithic application where all the functionality is tightly integrated into a single codebase, microservices break down the application into smaller, loosely coupled services.

Can be written in a variety of programming languages, and frameworks, and each service acts as a mini-application on its own.

- A small, loosely coupled service that is designed to perform a specific business function and each microservice can be developed, deployed, and scaled independently.
- **How do Microservices work?**
- Each microservice handles a particular business feature, like user authentication or product management, allowing for specialized development.
- Services interact via APIs, facilitating standardized information exchange and integration.
- Different technologies can be used for each service, enabling teams to select the best tools for their needs.
- Microservices can be updated independently, reducing risks during changes and enhancing system resilience.

- **Main components of Microservices Architecture**

- Main components of microservices architecture include:
- **Microservices:** Small, loosely coupled services that handle specific business functions, each focusing on a distinct capability.
- **API Gateway:** Acts as a central entry point for external clients also they manage requests, authentication and route the requests to the appropriate microservice.
- **Service Registry and Discovery:** Keeps track of the locations and addresses of all microservices, enabling them to locate and communicate with each other dynamically.
- **Load Balancer:** Distributes incoming traffic across multiple service instances and prevent any of the microservice from being overwhelmed.
- **Containerization:** Docker encapsulate microservices and their dependencies and orchestration tools like Kubernetes manage their deployment and scaling.
- **Event Bus/Message Broker:** Facilitates communication between microservices, allowing pub/sub asynchronous interaction of events between components/microservices.
- **Database per Microservice:** Each microservice usually has its own database, promoting data autonomy and allowing for independent management and scaling.
- **Caching:** Cache stores frequently accessed data close to the microservice which improved performance by reducing the repetitive queries.
- **Fault Tolerance and Resilience Components:** Components like circuit breakers and retry mechanisms ensure that the system can handle failures gracefully, maintaining overall functionality.

- **Real-World Example of Microservices**

- Let's understand the Microservices using the real-world example of Amazon E-Commerce Application:
- Amazon's online store runs on many small, specialized microservices, each handling a specific task. Working together, they create a smooth shopping experience.

- Below are the microservices involved in Amazon E-commerce Application:
- **User Service:** Handles user accounts and preferences, making sure each person has a personalized experience.
- **Search Service:** Helps users find products quickly by organizing and indexing product information.
- **Catalog Service:** Manages the product listings, ensuring all details are accurate and easy to access.
- **Cart Service:** Lets users add, remove, or change items in their shopping cart before checking out.
- **Wishlist Service:** Allows users to save items for later, helping them keep track of products they want.
- **Order Taking Service:** Processes customer orders, checking availability and validating details.
- **Order Processing Service:** Oversees the entire fulfillment process, working with inventory and shipping to get orders delivered.
- **Payment Service:** Manages secure transactions and keeps track of payment details.
- **Logistics Service:** Coordinates everything related to delivery, including shipping costs and tracking.
- **Warehouse Service:** Keeps an eye on inventory levels and helps with restocking when needed.
- **Notification Service:** Sends updates to users about their orders and any special offers.
- **Recommendation Service:** Suggests products to users based on their browsing and purchase history

How to migrate from Monolithic to Microservices Architecture?

- Step 1:** Begin by evaluating your current monolithic application. Identify its components and determine which parts can be shifted to microservices.
- Step 2:** Break down the monolith into specific business functions. Each microservice should represent a distinct capability that aligns with your business needs.
- Step 3:** Implement the Strangler Pattern to gradually replace parts of the monolithic application with microservices. This method allows for a smooth migration without a complete transition at once.
- Step 4:** Establish clear APIs and contracts for your microservices. This ensures they can communicate effectively and interact seamlessly.
- Step 5:** Create Continuous Integration and Continuous Deployment (CI/CD) pipelines. This automates testing and deployment, enabling faster and more reliable releases.

- **Step 6:** Introduce mechanisms for service discovery so that microservices can dynamically locate and communicate with each other, enhancing flexibility.
- **Step 7:** Set up centralized logging and monitoring tools. This provides insights into the performance of your microservices, helping to identify and resolve issues quickly.
- **Step 8:** Ensure consistent management of cross-cutting concerns, such as security and authentication, across all microservices to maintain system integrity.
- **Step 9:** Take an iterative approach to your microservices architecture. Continuously refine and expand your services based on feedback and changing requirements

Applications of Microservices

- **E-commerce platforms:** Separate services for inventory, payments, orders, and user management.
- **Banking & FinTech:** Independent services for accounts, transactions, fraud detection, and customer support.
- **Streaming services (e.g. Netflix, Spotify):** Services for user profiles, recommendations, content delivery, and billing.
- **Travel & Booking systems:** Services for flights, hotels, payments, and notifications.
- **Healthcare systems:** Patient records, appointment scheduling, billing, and reporting as separate services.
- **Social media platforms:** Microservices for feed, chat, notifications, and user profiles.

Benefits and Challenges of using Microservices Architecture

- **Benefits of Using Microservices Architecture**

- Teams can work on different microservices simultaneously.
- Issues in one service do not impact others, enhancing reliability.
- Each service can be scaled based on its specific needs.
- The system can quickly adapt to changing workloads.
- Teams can choose the best tech stack for each microservice.
- Small, cross-functional teams work independently.

- **Challenges of using Microservices Architecture**

- Managing service communication, network latency, and data consistency can be difficult.
- Decomposing an app into microservices adds complexity in development, testing and deployment.
- Network communication can lead to higher latency and complicates error handling.
- Maintaining consistent data across services is challenging, and distributed transactions can be complex.

Microservices vs Monolithic

Aspect	Microservices Architecture	Monolithic Architecture
Architecture Style	Decomposed into small, independent services.	Single, tightly integrated codebase.
Development Team Structure	Small, cross-functional teams for each microservice.	Larger, centralized development team.
Scalability	Independent scaling of individual services.	Scaling involves replicating the entire application.
Deployment	Independent deployment of services.	Whole application is deployed as a single unit.
Resource Utilization	Efficient use of resources as services can scale independently.	Resources allocated based on the overall application's needs.

Development Speed	Faster development and deployment cycles.	Slower development and deployment due to the entire codebase.
Flexibility	Easier to adopt new technologies for specific services.	Limited flexibility due to a common technology stack.
Maintenance	Easier maintenance of smaller, focused codebases.	Maintenance can be complex for a large, monolithic codebase.

Assignment

Architecture Evolution Case Study

- Each group selects one tech company (Netflix, Uber, Amazon, etc.).
- Research how they evolved from Monolithic → Microservices.
- Present in 5–7 slides with visuals and challenges faced during migration.

Thank you